

1. The washing machine has two operations: washing and dehydrating. Please write the *WashCommand* class and the *DehydrateCommand* class that implement the *Command* interface based on the command pattern. The *Command* interface contains one method named *executeCommand()*. Design the *Invoker* class and the *WashingMachine* class. The *Invoker* class contains two methods *setCommand()* and *invokeCommand()*. The *WashingMachine* class contains two methods *performWash()* and *performDehydrate()*. Write a *Client* class to test the result.

Client

```
package experiment17_1;

public class Client {

    public static void main(String[] args) {

        WashingMachine washingMachine = new
WashingMachine();

        Command washCommand = new
WashCommand(washingMachine);

        Command dehydrateCommand = new
DehydrateCommand(washingMachine);

        Invoker invoker = new Invoker();

        invoker.setCommand(washCommand);

        invoker.invokeCommand();

        invoker.setCommand(dehydrateCommand);

        invoker.invokeCommand();
```

```
}  
}
```

Command

```
package experiment17_1;  
  
interface Command {  
    void executeCommand();  
}
```

DehydrateCommand

```
package experiment17_1;  
  
class DehydrateCommand implements Command{  
    private WashingMachine washingMachine;  
    public DehydrateCommand(WashingMachine  
washingMachine){  
        this.washingMachine = washingMachine;  
    }  
  
    @Override  
    public void executeCommand() {  
        washingMachine.performDehydrate();  
    }  
}
```

Invoker

```
package experiment17_1;

class Invoker {
    private Command command;

    public void setCommand(Command command){
        this.command = command;
    }

    public void invokeCommand(){
        if(command != null){
            command.executeCommand();
        }else{
            System.out.println("No command set.");
        }
    }
}
```

WashCommand

```
package experiment17_1;

class WashCommand implements Command{
    private WashingMachine washingMachine;

    public WashCommand(WashingMachine
washingMachine){
```

```

        this.washingMachine = washingMachine;
    }

    @Override
    public void executeCommand() {
        washingMachine.performWash();
    }
}

```

WashingMachine

```

package experiment17_1;

class WashingMachine {
    public void performWash(){
        System.out.println("Performing wash operation...");
    }
    public void performDehydrate(){
        System.out.println("Performing dehydrate
operation...");
    }
}

```

Output

```
D:\JDK\bin\java.exe "-javaagent:D:\idea\IntelliJ\bin\idea_rt.jar" -jar D:\JDK\bin\java.exe
```

```
Performing wash operation...
```

```
Performing dehydrate operation...
```

```
Process finished with exit code 0
```